

Углубленный Python

Лекция 1

Тестирование

Кандауров Геннадий



education

Напоминание отметиться на портале

+ ОСТАВИТЬ ОТЗЫВ

vk образование

БлогиЛюдиПрограммаВакансииРасписание

Q

VK

Техно

Открыт приём заявок!

чт, 8 сентября

Нет занятий

пт, 9 сентября

18:00 Углубленный Py... с3
Введение в Python, основные
понятия, тестирование
Г. Кандауров

сб, 10 сентября

Нет занятий

вс, 11 сентября

Нет занятий

пн, 12 сентября

Нет занятий

Углубленный Python

↓ 0 ↑

Блог для материалов по курсу "Углубленный Python"

57 читателей, 2 топика

ПодписатьсяСоздать топик

Поиск по авторам, заголовку и тексту топика...Найти

Добро пожаловать на курс!

Углубленный Python

ИзменитьУдалить

Всем привет и добро пожаловать на курс по углубленному изучению Python!

Прямой эфир

МоиВсе

Геннадий Кандауров час назад
Углубленный Python → Добро пожаловать
на курс! 0

Екатерина Черкасова 7 дней назад
Стажировка → Приглашаем мобильных,
фронтенд- и бэкэнд-разработчиков на
Weekend Offer! 0

Дарья Вовченко 9 дней назад
Углубленный Python → Добро пожаловать
в образовательные проекты VK
Образование! 0

Дарья Вовченко 9 дней назад
Разработка веб-сервисов на

План курса

1. Тестирование
2. Повторение первого семестра
3. ООП, объектная модель
4. ООП, метапрограммирование, дескрипторы
5. АВС, стандартная библиотека
6. Потоки, процессы, IPC
7. Потоки, процессы, IPC - 2
8. Асинхронное программирование
9. Асинхронное программирование - 2
10. Устройство памяти
11. Расширения на C
12. Оформление кода, аннотация типов
13. FastAPI
14. Пробное интервью
15. Хакатон
16. Экзамен/пересдача

Контроль

Всего баллов - 100:

- **Домашние задания (ДЗ)** - 60 баллов

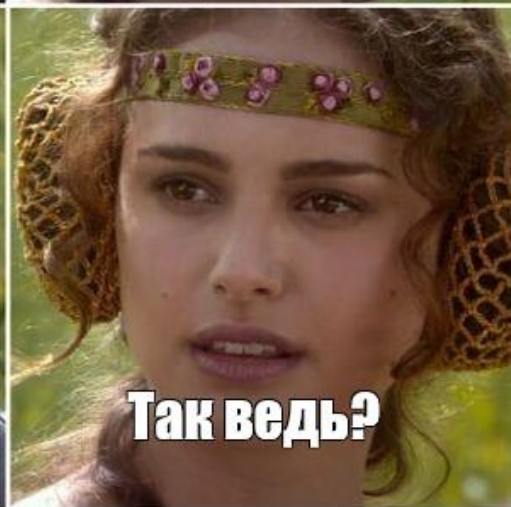
Квизы - 11 баллов

можно получить один балл в домашку за все правильные ответы в квизе

- **Хакатон** - 10 баллов
- **Экзамен** - 30 баллов

Итоговая оценка (по системе МГТУ):

- оценка **5** за **85** баллов и выше;
- оценка **4** за **70** баллов и выше;
- оценка **3** за **55** баллов и выше;
- оценка **2** за **54** балла и ниже;



Домашние задания

- **десять** ДЗ
- **семь** баллов до мягкого дедлайна
- **четыре** балла после мягкого дедлайна
- мягкий дедлайн - **две недели** после выдачи ДЗ, указан на портале
- **три** попытки, в зачет идет лучшая
- подбор тестовых сценариев - часть задания
- без списываний - баллы за домашку будут обнуляться

На оценку влияют:

1. Соответствие решения заданию
2. Корректность решения
3. Тесты:
важные случаи, граничные условия и тд
4. Оформление решения
5. Код решения как таковой
6. Дедлайны

Домашние задания

Сдача ДЗ ([подробнее в репе bmstu_python_s02_autumn_2026](#)):

- репозиторий должен быть приватным
- ветка main
- каждое ДЗ располагается в каталоге по номеру ДЗ без префиксов или суффиксов:
"01", "02", ... "10"
например, каталог для первой домашки должен называться "01";
- вспомогательные каталоги внутри каталога решения (**01**, **02**) не нужны, то есть все файлы и модули одной домашки должны находиться в корневом каталоге этой домашки (**01**, **02**);

Конкурс мемов

Я: Mam, a можно мне ML в продакшн?

Мама: Нет, у нас есть ML в продакшн дома!

ML в продакшн дома:

3. Тесты в отдельном модуле для каждого пункта

```
class SomeModel:
    def predict(self, message: str) -> float:
        # реализация не важна

def predict_message_mood(
    message: str,
    model: SomeModel,
    bad_thresholds: float = 0.3,
    good_thresholds: float = 0.8,
) -> str:
    ...
```



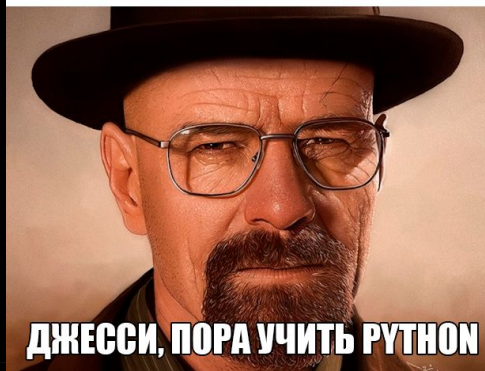
```
class ClassMeth:
    name = "cls_name"

    @classmethod
    def init_42(cls):
        print(f"Class {cls}")

    def __init__(self, val):
        self.val = val

    def inst_meth(self):
        print(f"Instance: {self.val}")
```

ClassMeth(42)

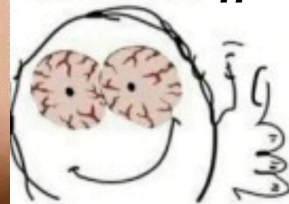


Как спят студенты?



Как спят преподаватели?

А НИКАК До 4 утра дз проверяют



Содержание занятия

1. Основные понятия, функции, классы
2. Тестирование
3. Пайплайн

Python



Python

Интерпретируемый язык с динамической и утиной типизацией, автоматической сборкой мусора и GIL.

Python - название спецификации языка

Реализации:

- CPython
- IronPython
- Jython
- PyPy

<https://github.com/python/cpython>

<https://www.python.org/doc/>



Все есть объект





Objects are Python's abstraction for data. All data in a Python program is represented by objects or by relations between objects.

docs.python.org

PyObject

```
typedef struct _object {  
    _PyObject_HEAD_EXTRA  
    Py_ssize_t ob_refcnt;  
    PyTypeObject *ob_type;  
} PyObject;
```

Объект

1. Каждый объект имеет идентичность, тип, значение
2. **id** никогда не меняется после создания объекта (**is** сравнивает **id** объектов)
3. Тип объекта определяет какие операции с ним можно выполнять
4. Значение объекта может меняться

Функции, lambda-функции

```
def add(a, b):  
    return a + b
```

```
def do_action(action, *args, **kwargs):  
    print(f"do {action} with {args}, {kwargs}")
```

```
    if callable(action):  
        return action(*args, **kwargs)  
    else:  
        return action
```

```
do_action(add, 1, b=2)
```

```
do_action(lambda x, y: x * y, 5, y=6)
```

Классы

```
class A:
    cls_attr = 42

    def __init__(self, val):
        self.val = val
        self._protected = 10
        self.__private_val = 20

    def print_name(self):
        print(self.__class__.__name__)

    @property
    def private_val(self):
        return self.__private_val
```

```
class B(A):
    cls_attr = 55

    def __init__(self, bval, val):
        super().__init__(val)

        self.bval = val
        self.__private_val = 999

b = B(1, 2)
print(b.private_val) # ???

isinstance(b, (int, A)) # ???
issubclass(B, object) # ???
```

Модули и пакеты

Модули являются основным компонентом организации кода в питоне (и это тоже объекты), объединяются в пакеты.

```
import this
collections = importlib.import_module('collections')
itertools = __import__('itertools')
```

file.py - модуль

--app/ - пакет

|--__init__.py

|--hello.py

Валидные импорты

```
import sys
```

```
from sys import path
```

```
from sys import *
```

```
import sys as s
```

```
from sys import path as sys_path
```

Тестирование





*Тестирование показывает
присутствие ошибок, а не их
отсутствие.*

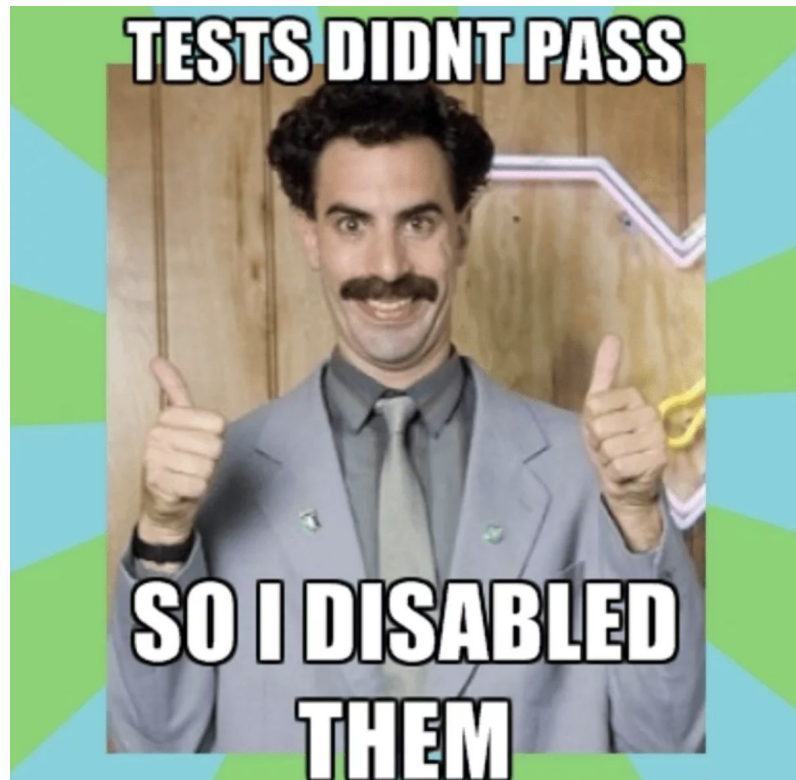
Эдсгер Дейстра

Тестированием можно доказать неправильность программы, но нельзя доказать её правильность.

Цели тестирования

- Проверка правильности реализации
- Проверка обработки внештатных ситуаций и граничных условий
- Минимизация последствий
- Подготовка ко внесению изменений
- Документирование кода

Цели тестирования



Виды тестирования

- Unit-тесты (модульные тесты)
- Функциональное тестирование
- Системное тестирование
- Интеграционное тестирование
- Регрессионное тестирование
- Маркер тесты
- Тестирование производительности
 - Нагрузочное
 - Стресс

TDD

TDD (Test Driven Development) – техника разработки ПО, основывается на повторении коротких циклов разработки: пишется тест, покрывающий желаемое изменение, затем пишется код, который позволит пройти тест, и далее проводится рефакторинг нового кода.

Степень покрытия тестами

coverage - библиотека для проверки покрытия тестами.

```
pip install coverage
```

```
coverage run tests.py
```

```
coverage report -m
```

```
coverage html
```

Инструменты тестирования в Python

- `doctest`
- `unittest`
- `pytest`
- `factory_boy`

doctest

```
def multiply(a, b):  
    """  
    >>> multiply(4, 3)  
    12  
    >>> multiply("a", 3)  
    'aaa'  
    """  
    return a * b
```

```
python -m doctest <file>
```

unittest

```
class TestCase
```

- `def setUp(self):`
установки запускаются перед каждым тестом
- `def tearDown(self):`
очистка после каждого метода
- `def test_<название теста>(self):`
код теста
- `def test_check_value(self):`
код теста

unittest: TestCase

```
import unittest

class TestString(unittest.TestCase):

    def test_upper(self):

        self.assertEqual("text".upper(), "TEXT")

        assert "text".upper() == "TEXT"

if __name__ == "__main__":

    unittest.main()
```

unittest: набор assert*

- `assertEqual(a, b)`
- `assertNotEqual(a, b)`
- `assertTrue(x)`
- `assertFalse(x)`
- `assertIsNone(x)`
- `assertIs(a, b)`
- `assertIsNot(a, b)`
- `assertIn(a, b)`
- `assertIsInstance(a, b)`
- `assertLessEqual(a, b)`
- `assertListEqual(a, b)`
- `assertDictEqual(a, b)`
- `assertRaises(exc, fun, *args, **kwargs)`

unittest: mock

Mock — это объект-пустышка, который заменяет некий реальный объект (функцию, экземпляр) для определенной части программы.

- Высокая скорость
- Избежание нежелательных побочных эффектов во время тестирования
- Позволяет задать специальное поведение в рамках теста

```
from unittest.mock import patch
```

```
class TestUserSubscription(TestCase):  
    @patch("users.views.get_status", return_value=True)  
    def test_subscription(self, get_status_mock):  
        ...
```

unittest: mock

Атрибуты объекта Mock с информацией о вызовах

- `called` — вызывался ли объект вообще
- `call_count` — количество вызовов
- `call_args` — аргументы последнего вызова
- `call_args_list` — список всех аргументов
- `method_calls` — аргументы обращений к вложенным методам и атрибутам
- `mock_calls` — то же самое, но в целом и для самого объекта, и для вложенных

```
self.assertEqual(get_subscription_status_mock.call_count, 1)
```

unittest: запуск тестов

Найти и выполнить все тесты

```
python -m unittest discover
```

Тесты нескольких модулей

```
python -m unittest test_module1 test_module2
```

Тестирование одного кейса - набора тестов

```
python -m unittest tests.SomeTestCase
```

Тестирование одного метода

```
python -m unittest tests.SomeTestCase.test_some_method
```

factory_boy

Библиотека `factory_boy` служит для генерации разнообразных объектов (в т.ч. связанных) по заданным параметрам.

<https://factoryboy.readthedocs.io/en/stable/>

<https://faker.readthedocs.io/en/master/>

```
pip install factory_boy
```

Домашнее задание #01

- Реализовать предиктор и фильтр-генератор
- Написать тесты `unittest` или `pytest`
- Проверить покрытие тестов через `coverage`
- Проверить и поправить код `flake8` и `pylint`
- Настроить пайплайн в `github`
- Зеленый пайплайн в `github`

Напоминание отметиться на портале Vol 2

+ ОСТАВИТЬ ОТЗЫВ ПОСЛЕ ЛЕКЦИИ

vk образование

БлогиЛюдиПрограммаВакансииРасписание

Q<

VK

Техно

Открыт приём заявок!

чт, 8 сентября

Нет занятий

пт, 9 сентября

18:00 Углубленный Py... с3
Введение в Python, основные
понятия, тестирование
Г. Кандауров

сб, 10 сентября

Нет занятий

вс, 11 сентября

Нет занятий

пн, 12 сентября

Нет занятий

Углубленный Python

↓ 0 ↑

Блог для материалов по курсу "Углубленный Python"

57 читателей, 2 топика

ПодписатьсяСоздать топик

Поиск по авторам, заголовку и тексту топика...

Найти

Добро пожаловать на курс!

Углубленный Python

ИзменитьУдалить

Всем привет и добро пожаловать на курс по углубленному изучению Python!

Прямой эфир

МоиВсе

Геннадий Кандауров час назад
Углубленный Python → Добро пожаловать
на курс! 0

Екатерина Черкасова 7 дней назад
Стажировка → Приглашаем мобильных,
фронтенд- и бэкэнд-разработчиков на
Weekend Offer! 0

Дарья Вовченко 9 дней назад
Углубленный Python → Добро пожаловать
в образовательные проекты VK
Образование! 0

Дарья Вовченко 9 дней назад
Разработка веб-сервисов на

Спасибо за
внимание

